Normal form for effect axioms

Suppose there are two positive effect axioms for the fluent *Broken:*

$$\text{Fragile}(x) \supset \text{Broken}(x, do(\text{drop}(r,x), s))$$

$$\text{NextTo}(b,x,s) \supset \text{Broken}(x, do(\text{explode}(b), s))$$

These can be rewritten as

$$\exists r \{a = \text{drop}(r,x) \wedge \text{Fragile}(x)\} \vee \exists b \{a = \text{explode}(b) \wedge \text{NextTo}(b,x,s)\} \supset \text{Broken}(x, do(a,s))$$

Similarly, consider the negative effect axiom:

$$\neg\text{Broken}(x, do(\text{repair}(r,x), s))$$

which can be rewritten as

$$\exists r \{a = \text{repair}(r,x)\} \supset \neg\text{Broken}(x, do(a,s))$$

In general, for any fluent F, we can rewrite all the effect axioms as as two formulas of the form

$$P_F(\mathbf{x}, a, s) \supset F(\mathbf{x}, do(a,s)) \quad (1)$$

$$N_F(\mathbf{x}, a, s) \supset \neg F(\mathbf{x}, do(a,s)) \quad (2)$$

where $P_F(\mathbf{x}, a, s)$ and $N_F(\mathbf{x}, a, s)$ are formulas whose free variables are among the x_i, a, and s.

Explanation closure

Now make a completeness assumption regarding these effect axioms:

assume that (1) and (2) characterize *all* the conditions under which an action a changes the value of fluent F.

This can be formalized by explanation closure axioms:

$$\neg F(\mathbf{x}, s) \wedge F(\mathbf{x}, do(a,s)) \supset \mathrm{P}_F(\mathbf{x}, a, s) \qquad (3)$$

if F was false and was made true by doing action a then condition P_F must have been true

$$F(\mathbf{x}, s) \wedge \neg F(\mathbf{x}, do(a,s)) \supset \mathrm{N}_F(\mathbf{x}, a, s) \qquad (4)$$

if F was true and was made false by doing action a then condition N_F must have been true

These explanation closure axioms are in fact disguised versions of frame axioms!

$$\neg F(\mathbf{x}, s) \wedge \neg \mathrm{P}_F(\mathbf{x}, a, s) \supset \neg F(\mathbf{x}, do(a,s))$$

$$F(\mathbf{x}, s) \wedge \neg \mathrm{N}_F(\mathbf{x}, a, s) \supset F(\mathbf{x}, do(a,s))$$

Successor state axioms

Further assume that our KB entails the following

- integrity of the effect axioms: $\neg\exists\, \boldsymbol{x}, a, s.\ \mathrm{P}_F(\boldsymbol{x}, a, s) \wedge \mathrm{N}_F(\boldsymbol{x}, a, s)$
- unique names for actions:

$$A(x_1,...,x_n) = A(y_1,...,y_n) \supset (x_1=y_1) \wedge ... \wedge (x_n=y_n)$$
$$A(x_1,...,x_n) \neq B(y_1,...,y_m) \quad \text{where } A \text{ and } B \text{ are distinct}$$

Then it can be shown that KB entails that (1), (2), (3), and (4) together are logically equivalent to

$$F(\boldsymbol{x}, do(a,s)) \equiv \mathrm{P}_F(\boldsymbol{x}, a, s) \vee (F(\boldsymbol{x}, s) \wedge \neg\mathrm{N}_F(\boldsymbol{x}, a,s))$$

This is called the successor state axiom for F.

For example, the successor state axiom for the *Broken* fluent is:

$$\begin{aligned} &\mathrm{Broken}(x, do(a,s)) \equiv \\ &\quad \exists r\, \{a=\mathrm{drop}(r,x) \wedge \mathrm{Fragile}(x)\} \\ &\vee\ \exists b\, \{a=\mathrm{explode}(b) \wedge \mathrm{NextTo}(b,x,s)\} \\ &\vee\ \mathrm{Broken}(x, s) \wedge \neg\exists r\, \{a=\mathrm{repair}(r,x)\} \end{aligned}$$

An object x is broken after doing action a
iff
a is a dropping action and x is fragile,
or a is a bomb exploding
where x is next to the bomb,
or x was already broken and
a is not the action of repairing it

Note universal quantification: for *any* action a ...

A simple solution to the frame problem

This simple solution to the frame problem (due to Ray Reiter) yields the following axioms:

- one successor state axiom per fluent
- one precondition axiom per action
- unique name axioms for actions

Moreover, we do not get fewer axioms at the expense of prohibitively long ones

> the length of a successor state axioms is roughly proportional to the number of actions which affect the truth value of the fluent

The conciseness and perspicuity of the solution relies on

- quantification over actions
- the assumption that relatively few actions affect each fluent
- the completeness assumption (for effects)

Moreover, the solution depends on the fact that actions always have deterministic effects.

Limitation: primitive actions

As yet we have no way of handling in the situation calculus complex actions made up of other actions such as

- conditionals: If the car is in the driveway then drive else walk
- iterations: while there is a block on the table, remove one
- nondeterministic choice: pickup up some block and put it on the floor

and others

Would like to *define* such actions in terms of the primitive actions, and inherit their solution to the frame problem

Need a compositional treatment of the frame problem for complex actions

Results in a novel programming language for discrete event simulation and high-level robot control

The Do formula

For each complex action A, it is possible to define a formula of the situation calculus, $Do(A, s, s')$, that says that action A when started in situation s may legally terminate in situation s'.

Primitive actions: $Do(A, s, s') = Poss(A,s) \wedge s'=do(A,s)$

Sequence: $Do([A;B], s, s') = \exists s''. Do(A, s, s'') \wedge Do(B, s'', s')$

Conditionals: $Do([\textbf{if } \phi \textbf{ then } A \textbf{ else } B], s, s') =$
$\phi(s) \wedge Do(A, s, s') \vee \neg\phi(s) \wedge Do(B, s, s')$

Nondeterministic branch: $Do([A \mid B], s, s') = Do(A, s, s') \vee Do(B, s, s')$

Nondeterministic choice: $Do([\pi x. A], s, s') = \exists x. Do(A, s, s')$

etc.

Note: programming language constructs with a purely logical situation calculus interpretation

GOLOG

GOLOG (Algol in logic) is a programming language that generalizes conventional imperative programming languages

- the usual imperative constructs + concurrency, nondeterminism, more...
- bottoms out *not* on operations on internal states (assignment statements, pointer updates) but on *primitive actions* in the world (*e.g.* pickup a block)
- what the primitive actions do is user-specified by precondition and successor state axioms

What does it mean to "execute" a GOLOG program?

- find a sequence of primitive actions such that performing them starting in some initial situation s would lead to a situation s' where the formula $Do(A, s, s')$ holds
- give the sequence of actions to a robot for actual execution in the world

Note: to find such a sequence, it will be necessary to reason about the primitive actions

A ; ***if*** Holding(x) ***then*** B ***else*** C

to decide between B and C we need to determine if the fluent *Holding* would be true after doing A

GOLOG example

Primitive actions: pickup(x), putonfloor(x), putontable(x)

Fluents: Holding(x,s), OnTable(x,s), OnFloor(x,s)

Action preconditions: $Poss(\text{pickup}(x), s) \equiv \forall z.\neg\text{Holding}(z, s)$

$Poss(\text{putonfloor}(x), s) \equiv \text{Holding}(x, s)$

$Poss(\text{putontable}(x), s) \equiv \text{Holding}(x, s)$

Successor state axioms:

$\text{Holding}(x, do(a,s)) \equiv a=\text{pickup}(x) \vee$
$\text{Holding}(x,s) \wedge a\neq\text{putontable}(x) \wedge a\neq\text{putonfloor}(x)$

$\text{OnTable}(x, do(a,s)) \equiv a=\text{putontable}(x) \vee \text{OnTable}(x,s) \wedge a\neq\text{pickup}(x)$

$\text{OnFloor}(x, do(a,s)) \equiv a=\text{putonfloor}(x) \vee \text{OnFloor}(x,s) \wedge a\neq\text{pickup}(x)$

Initial situation: $\forall x.\neg\text{Holding}(x, S_0)$

$\text{OnTable}(x, S_0) \equiv x=\text{A} \vee x=\text{B}$

Complex actions:

proc ClearTable : ***while*** $\exists b.\text{OnTable}(b)$ ***do*** $\pi b\ [\text{OnTable}(b)?\ ;\ \text{RemoveBlock}(b)]$

proc RemoveBlock(x) : pickup(x) **;** putonfloor(x)

Running GOLOG

To find a sequence of actions constituting a legal execution of a GOLOG program, we can use Resolution with answer extraction.

For the above example, we have

$$KB \models \exists s.\ Do(\text{ClearTable}, S_0, s)$$

The result of this evaluation yields

$$s = do(\text{putonfloor}(\text{B}), do(\text{pickup}(\text{B}), do(\text{putonfloor}(\text{A}), do(\text{pickup}(\text{A}), S_0))))$$

and so a correct sequence is

$$\langle \text{pickup(A), putonfloor(A), pickup(B), putonfloor(B)} \rangle$$

When what is known about the actions and initial state can be expressed as Horn clauses, the evaluation can be done in Prolog.

The GOLOG interpreter in Prolog has clauses like

```
do(A,S1,do(A,S1)) :- prim_action(A), poss(A,S1).
do(seq(A,B),S1,S2) :- do(A,S1,S3), do(B,S3,S2).
```

This provides a convenient way of controlling a robot at a high level.

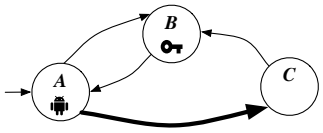
SAPIENZA Università di Roma – MSc. in Engineering in Computer Science

Artificial Intelligence & Machine Learning – June 23, 2022

(Time to complete the test: 2:30 hours)

Part 1 - Artificial Intelligence *(Time to complete Part 1: 100 minutes)*

Consider the following scenario. A robot, *self*, can move across a network of locations shown in the picture below.



To traverse some edges of the network *self* needs a key (specifically the edge in **bold** in the picture).

Non-fluents predicates: $Location(x)$ denoting locations, in our case A, B, C; $Obj(x)$ denoting the objects, in our case only the key key; $connected(x, y)$ where x and y are locations, denoting the edges not requiring a key in the network (see picture above); $connectedKey(x, y)$ where x and y are locations, denoting the edges that need the key to be traversed (see the picture above).

Fluents: $SelfAt(x)$ where x is a location, denoting that *self* is at location x; $At(x, y)$ where x is an object and y a location, denoting that the object x is at location y; $SelfHas(x)$ where x is an object, and denotes that *self* has object x with himself/herself.

Actions as available to *self*:

- $goto(\ell_1, \ell_2)$ to move from location ℓ_1 to location ℓ_2. To do so *self* needs to be in location ℓ_1, moreover if traversing the edge in the network needs the key, *self* need to have the key. The effect of $goto(\ell_1, \ell_2)$ is that *the* new location of *self*, together with all the objects that *self* has with him/her (i.e., the key), will be ℓ_2.
- $pickup(obj)$ to pick up object obj (i.e., the key). To do so *self* needs to be in the same location as obj. The effect of $pickup(obj)$ is that *self* will have obj with himself/herself.
- ~~$drop(obj)$ to drop obj (i.e., the key). To do so *self* needs to have obj with himself/herself. The effect of $drop(obj)$ is that *self* will not have obj with himself/herself and the location of obj will be that of *self*.~~

Initially: *self* is at location A, the key is at location B, and *self* does not have any object with himself/herself. (The non-fluent predicates have their extension as above.)

Exercise 1. Formalize the above scenario as a Basic Action Theory in Reiter's Situation Calculus. Then check by regression:

1. Whether the action $goto(A, C)$ is executable in S_0;
2. Whether the sequence of actions $goto(A, B); pickup(key); goto(B, A); goto(A, C)$ is executable in S_0;
3. Whether, after executing the sequence of actions $goto(A, B); pickup(key); goto(B, A); goto(A, C)$ in S_0, *self* has the key with himself/herself.

Exercise 2. Considering as goal $SelfAt(C)$, formalize the above scenario as a PDDL domain file and PDDL problem file. Then:

1. Draw the corresponding transition system;
2. Solve planning for achieving the above goal by using backward fixpoint computation, reporting the various stages of the fixpoint computation;
3. Solve planning for achieving the above goal by using forward depth-first search (uniformed), reporting the various steps of the forward search computation.

Exercise 3. Check, by using tableaux, whether the following FOL formula is indeed valid, and if not show a counterexample: $(\,(\forall x.(A(x) \supset B(x))\,) \;\equiv\; (\,(\forall y.A(y)) \supset (\forall z.B(z))\,)$.

Exercise 4. Describe the DPLL procedure for satisfiability and use it to find a model (a satisfying interpretation) of the following set of clauses: $\{\ \{A, B, \neg C\}, \{\neg A, B, C\}, \{\neg A\}, \{\neg B, C\}\ \}$.

SAPIENZA Università di Roma–MSc. in Engineering in Computer Science
Artificial Intelligence and Machine Learning
Part 2: Machine Learning

23 June 2022

- **Time Limits**: AI+ML: 2h 30m; AI only: 1h 40m; ML only: 50m.
- **Important:** The AI and ML parts must be delivered separately (one set of sheets for each part). Write your name and matricola number on every delivered sheet.

Exercise 5

You need to devise a system for oil-price prediction. The available data include N instances, each representing a sequence of average prices for 11 consecutive days. That is, the generic instance has the form $x = (d_1, \ldots, d_{11})$, where d_i represents the average oil price of the i-th day in the sequence.

1. Design a neural network that can predict the average price of one day, given that of the previous consecutive 10 days. The ANN must include two hidden layers (plus one output layer). The first hidden layer contains 30 units. The design must specify: the dimensions of the weight matrices W_1, W_2 and W_3 of the two hidden and the output layers, respectively, and the type of units used in each layer. Bias weights can be ignored.
2. Given the design above, indicate the number of units of the second hidden and the output layers, and the number of trainable parameters of the network.
3. Indicate one plausible algorithm for training the ANN (no need to report the pseudo-code) and explain how you would use the available N instances in the dataset to evaluate the performance of the system.

Exercise 6

1. Define the notion of Markov Decision Process (MDP). For each element of an MDP, provide its mathematical definition and an intuitive description (what they model). Indicate which elements are unknown in Reinforcement Learning.
2. Illustrate and explain the *Markov Assumption*.
3. Define the notions of *(expected) cumulative discounted reward* and *optimal policy*.

FOUNDATIONS OF THE SITUATION CALCULUS

Motivation

- An analogy: The Peano axioms for number theory.
- The second order language (with equality):
  - A single constant 0.
  - A unary function symbol σ (successor function).
  - A binary predicate symbol $<$.
- A fragment of Peano arithmetic:

$$\sigma(x) = \sigma(y) \supset x = y,$$

$$(\forall P).\{P(0) \wedge [(\forall x).P(x) \supset P(\sigma(x))]\} \supset (\forall x)P(x)$$

$$\neg x < 0,$$

$$x < \sigma(y) \equiv x \leq y.$$

  Here, $x \leq y$ is an abbreviation for $x < y \vee x = y$.
- The second sentence is a second order induction axiom. It is a second order way of characterizing the domain of discourse as the $smallest$ set such that
  1. 0 is in the set.
  2. Whenever x is in the set, so is $\sigma(x)$.
- Second order Peano arithmetic is $categorical$ (it has a unique model).

- First order Peano arithmetic: Replace the second order axiom by an induction $schema$ representing countably infinitely many first order sentences, one for each instance of P obtained by replacing P by a first order formula with one free variable.
- First order Peano arithmetic is not categorical; it has (infinitely many) $nonstandard$ models. This follows from the Gödel incompleteness theorem, which says that first order arithmetic is $incomplete$, i.e. there are sentences true of the principal interpretation of the first order axioms (namely, the natural numbers) which are false in some of the nonstandard models, and hence not provable from the first order axioms.
- So why not use the second order axioms? Because second order logic is incomplete, i.e. there is no "decent" axiomatization of second order logic which will yield all the valid second order sentences!
- So why appeal to second order logic at all? Because $semantically$, but not syntactically, it characterizes the natural numbers. We'll find the same phenomenon in semantically characterizing the situation calculus.

Foundational Axioms for the Situation Calculus

- We use a 3-sorted language: The sorts are $situation$, $object$ and $action$. There is a unique situation constant symbol, S_0, denoting the initial situation. It is like the number 0 in Peano arithmetic. Unlike Peano arithmetic which has a unique successor function, we have a family of successor functions.

  $do : action \times situation \rightarrow situation.$

- The axioms:

$$do(a_1, s_1) = do(a_2, s_2) \supset a_1 = a_2 \wedge s_1 = s_2 \quad (1)$$

$$(\forall P).P(S_0) \wedge (\forall a, s)[P(s) \supset P(do(a, s))] \supset (\forall s)P(s) \quad (2)$$

$$\neg s \sqsubset S_0, \quad (3)$$

$$s \sqsubset do(a, s') \equiv s \sqsubseteq s', \quad (4)$$

  where $s \sqsubseteq s'$ is an abbreviation for $s \sqsubset s' \vee s = s'$.

- Compare with the Peano axioms for the natural numbers.

- Axiom (2) is a second order way of limiting the sort $situation$ to the smallest set containing S_0, and closed under the application of the function do to an action and a situation. Any model of these axioms will have its domain of situations isomophic to the smallest set $\mathcal{S}$ satisfying:

  1. $S_0 \in \mathcal{S}$.